LOW-POWER ALU DESIGN USING VERILOG

Industry-Oriented VLSI Course Project

Prepared By: Vayunandan Mishra

Domain: VLSI Design | RTL Design | FPGA Design

Language: Verilog/SystemVerilog

Project Report

Abstract

This project presents the design and verification of an 8-bit Low-Power Arithmetic Logic Unit (ALU) using SystemVerilog. The ALU supports arithmetic, logical, shift, increment, decrement, and compare operations. Operand isolation is implemented as a low-power technique to reduce unnecessary switching activity. Functional verification is performed using a dedicated testbench and waveform analysis in GTKWave.

Chapter 1: Introduction

The Arithmetic Logic Unit is the computational core of modern processors. Every arithmetic and logical instruction executed by a CPU passes through the ALU. This project demonstrates digital design principles, combinational logic design, verification methodology, and power-aware RTL coding.

Chapter 2: Objectives

Design a synthesizable ALU, implement multiple arithmetic and logical operations, generate status flags, demonstrate operand isolation, verify functionality through simulation, and create a GitHub-ready portfolio project.

Chapter 3: ALU Theory

An ALU performs operations on binary operands. Arithmetic functions include addition and subtraction. Logical functions include AND, OR, XOR, and NOT. Shift operations move data left or right. Compare operations determine equality. Flags provide status information regarding results.

Chapter 4: Low-Power Design

Power consumption is a major concern in modern VLSI systems. Dynamic power is proportional to switching activity. Operand isolation prevents unnecessary signal transitions by disabling internal computation when enable is low. This reduces switching activity and demonstrates power-aware RTL design.

Chapter 5: Architecture

Inputs include A[7:0], B[7:0], Opcode[3:0], and Enable. The architecture consists of operand isolation logic, opcode decoder, ALU processing block, and flag generation logic. Outputs include Result[7:0], Zero, Carry, Overflow, and Negative flags.

Chapter 6: Opcode Table

0000-ADD, 0001-SUB, 0010-AND, 0011-OR, 0100-XOR, 0101-NOT, 0110-SHIFT LEFT, 0111-SHIFT RIGHT, 1000-INCREMENT, 1001-DECREMENT, 1010-COMPARE.

Chapter 7: RTL Design

The RTL is written using SystemVerilog. A parameterized design approach allows scalability. A case statement decodes opcodes and performs the selected operation. Arithmetic flags are generated for monitoring processor status.

Chapter 8: Testbench Design

A dedicated testbench applies stimulus vectors to verify all ALU functions. The testbench generates console logs and VCD waveform files for debugging and validation.

Chapter 9: Simulation Results

Simulation was performed using Icarus Verilog and GTKWave. Results verified correct operation of ADD, SUB, AND, OR, XOR, NOT, SHIFT LEFT, SHIFT RIGHT, INCREMENT, DECREMENT, and COMPARE functions.

Chapter 10: Waveform Analysis

Waveforms confirmed proper opcode decoding and result generation. The result bus changed correctly for each opcode. Operand isolation was verified by disabling enable and observing output suppression.

Chapter 11: FPGA Implementation

The design can be synthesized using Xilinx Vivado. Inputs may be mapped to switches and outputs to LEDs. Utilization, timing, and power reports can be generated even without physical hardware.

Chapter 12: Results and Discussion

The design successfully achieved all objectives. Functional verification passed. Low-power operand isolation worked correctly. The architecture is suitable for educational FPGA and ASIC design flows.

Chapter 13: Future Enhancements

Future improvements include clock gating, power gating concepts, assertions, functional coverage, self-checking testbenches, FPGA hardware validation, and advanced power analysis.

Chapter 14: Conclusion

The Low-Power ALU project successfully demonstrates RTL design, digital logic implementation, verification methodology, waveform analysis, and power-aware design techniques. The project is suitable for academic submission, GitHub portfolios, and VLSI internship interviews.

References

SystemVerilog IEEE Standard, Icarus Verilog Documentation, GTKWave Documentation, Xilinx Vivado User Guides, Digital Design and Computer Architecture textbooks.